ECSE 4320/6320 Advanced Computer Systems

Class-wide Projects, due date: October 16

Design Philosophy

These projects are designed for an environment in which students may use AI tools for coding assistance. The goal is not to ban AI, but to make the project harder to complete by blind code generation alone. Students are graded on correctness, controlled measurement, reproducibility, and architectural interpretation.

- AI assistance is allowed for coding, debugging, plotting, and writing support.
- Students remain responsible for understanding, validating, and explaining all submitted code, measurements, and claims.
- Polished code without validated methodology and credible analysis is insufficient.
- Each project must include at least one case where the expected optimization **does not** help or where a naive/AI-generated benchmark would be misleading.

Common Requirements for All Class-wide Projects

1. AI-use note: briefly state which tools were used, what they helped generate, and what was independently modified, tested, or rejected.
2. Reproducible repository: include source code, build scripts, run scripts, plotting scripts, raw data, processed data, and README.
3. Pre-experiment hypothesis: before reporting results, state what trend you expected and why.
4. Correctness validation: compare against a simple reference implementation or known sanity checks.
5. Measurement discipline: record hardware, OS, compiler/tool versions, frequency/governor, SMT state, pinning/isolation, repetitions, and error bars when feasible.
6. Negative/failure case: include at least one experiment showing when the technique stops helping or gives misleading results.
7. Honest anomalies: discuss unexpected results with plausible hypotheses rather than hiding them.

Optional GPU/CUDA Extension
Students or teams with access to a CUDA-capable GPU may add a CPU-vs-GPU comparison to a relevant project. This extension is optional and will not be required for full credit. GPU reports must separate kernel-only time from end-to-end time including host-device transfer where applicable, and must explain when GPU acceleration helps or fails.

Project #1: SIMD Advantage Profiling

Modern CPUs provide SIMD vector units such as SSE, AVX, AVX-512, or NEON that can process multiple data elements per instruction. This project quantifies real-world SIMD speedup for simple numeric kernels and explains when vectorization helps, compresses, or fails.

Learning Goals

- Measure scalar vs. SIMD speedup for common kernels under realistic conditions.
- Distinguish compute-bound and memory-bound regimes.
- Understand the impact of data type, alignment, stride/tail handling, and working-set size.
- Verify that SIMD actually occurred using compiler reports or disassembly.
- Interpret results using arithmetic intensity, measured memory bandwidth, and a roofline-style argument.

Scope and Tools

- Keep required experiments single-threaded to isolate SIMD effects.
- Use GCC or Clang with optimization enabled; record exact compiler versions and flags.
- Use compiler vectorization reports and/or targeted disassembly to verify vector instructions.
- Use reliable timing and statistical scripts; compute runtime, GFLOP/s or GiB/s, and cycles per element.
- Recommended: perf or similar counters for instructions, vector instructions, cache misses, and memory traffic.

Kernel Set

Choose at least three kernels:

- SAXPY / AXPY: y = a*x + y
- Dot product / reduction: s = sum(x_i*y_i)
- Elementwise multiply: z_i = x_i*y_i
- 1D three-point stencil: y_i = a*x_{i-1} + b*x_i + c*x_{i+1}

Experimental Knobs

- Data type: float32 vs. float64; optionally int32.
- Alignment and tail: aligned arrays vs. arrays whose starting address is intentionally shifted from a vector/cache-line boundary; sizes with and without vector-width remainders.
- Access pattern: unit stride vs. stride 2, 4, 8, or gather-like index patterns where meaningful.
- Working-set size: sweep N across L1, L2, LLC, and DRAM-resident regimes.
- Compiler/ISA flags: scalar-only, default auto-vectorized, and CPU-targeted widest available vectors.

Required Experiments and Plots

- Scalar baseline vs. auto-vectorized build for each selected kernel across working-set sizes. Report speedup and achieved throughput.
- Locality sweep for one kernel crossing cache levels. Annotate where SIMD gains compress as the workload becomes memory-bound.
- Alignment and tail-handling study. Quantify prologue/epilogue, unaligned load, or masking effects.
- Stride/gather study. Show how non-unit stride reduces effective bandwidth and SIMD efficiency.
- Data-type comparison. Explain results using lane count, arithmetic intensity, and operation cost.
- Vectorization verification. Include concise compiler-report or disassembly evidence; do not paste full dumps.
- Roofline interpretation for at least one kernel using measured memory bandwidth and estimated CPU peak FLOP rate.
- Include one negative example: either a loop that appears SIMD-friendly but is not actually vectorized, or a loop that is vectorized but shows little speedup because memory bandwidth dominates. Verify and explain the result.

Deliverables

- GitHub repository with code, build/run scripts, raw measurements, processed data, and plotting scripts.
- Methodology: CPU model and ISA, compiler and flags, OS, frequency policy, SMT state, timing method, repetitions, and data initialization.
- Plots/tables with units, legends, and variability/error bars when feasible.
- Short report connecting results to cache locality, vector width, arithmetic intensity, memory bandwidth, and anomalies.

Grading Rubric

Total: 110 points

Category	Points	Description
Baseline, correctness, reproducibility	15	Reference validation, controlled timing, reproducible scripts, and documented environment.
Vectorization verification	15	Clear evidence of SIMD and correct interpretation of vector width/FMA/masking.
Locality and memory-bound analysis	20	Working-set transitions and explanation of compressed SIMD gains.
Alignment, tail, stride/gather studies	20	Quantified effects and convincing explanation of access-pattern behavior.
Roofline and quantitative insight	20	Correct arithmetic intensity, measured bandwidth use, and compute/memory-bound conclusion.
AI-use transparency and failure case	10	Honest AI-use note and meaningful negative/misleading-example analysis.
Report quality	10	Clear plots, concise writing, and thoughtful limitations.

Project #2: Cache & Memory Performance Profiling

Modern CPUs use a deep hierarchy of L1/L2/L3 caches and DRAM. Each level differs in latency, bandwidth, capacity, and queuing behavior. This project measures the hierarchy and explains how locality, concurrency, cache misses, and TLB misses determine performance.

Learning Goals

- Measure zero-queue latency for L1, L2, L3, and DRAM where feasible.
- Measure DRAM bandwidth under different access granularities, strides, and read/write mixes.
- Reveal the throughput-latency tradeoff as memory access intensity grows.
- Correlate cache-miss and TLB-miss behavior with kernel performance.
- Use Little's Law and AMAT-style reasoning to interpret results.

Tools

- Intel Memory Latency Checker or an equivalent memory benchmark for latency/bandwidth sweeps.
- perf or equivalent counters for cache/TLB behavior, cycles, instructions, and memory events.
- Simple loop kernels, such as z[i] = x[i] * y[i] or y[i] = a*x[i] + y[i], used to correlate cache/TLB misses with runtime.

Experimental Knobs

- Access pattern: sequential vs. random; strides near 64B, 256B, and 1024B.
- Read/write ratio: 100% read, 100% write, 70/30, and 50/50 where supported.
- Access intensity: outstanding requests or threads, including loaded-latency sweeps.
- Working-set size: range from L1-resident to DRAM-resident.
- Page locality: 4KB pages vs. huge pages where available.

Required Experiments and Plots

- Zero-queue baselines: per-level latency table with units and CPU-frequency conversion where useful.
- Pattern and granularity sweep: latency and bandwidth across sequential/random and stride/granularity settings.
- Read/write mix sweep: compare throughput and latency under the required mixes.
- Intensity sweep: plot throughput vs. latency, identify the knee, and relate it to Little's Law.
- Working-set sweep: annotate L1/L2/LLC/DRAM transition regions.
- Cache-miss impact: vary footprint or access pattern and correlate perf counters with runtime.
- TLB-miss impact: vary page locality and evaluate huge pages where possible.
- Show one benchmark pitfall, such as accidentally measuring cache reuse instead of DRAM, failing to pin threads, or allowing frequency scaling to distort results.

Deliverables

- Scripts/commands, raw data, processed data, and plotting code.
- System configuration: CPU, memory, OS/kernel, tool versions, governor/frequency policy, SMT state, pinning, NUMA placement if relevant.
- Plots/tables with units, repeated trials, and error bars when feasible.
- Analysis tied to latency, bandwidth, concurrency, cache/TLB counters, AMAT, and Little's Law.

Grading Rubric

Total: 185 points

Category	Points	Description
Zero-queue baselines	25	Methodologically sound latency measurements with clear units and caveats.
Pattern/granularity/RW sweeps	35	Complete and well-controlled bandwidth/latency comparisons.
Throughput-latency knee	35	Clear knee identification, diminishing-return discussion, and Little's Law connection.

Category	Points	Description
Working-set/cache/TLB analysis	35	Counters and performance correlation used correctly.
Experimental rigor	25	Pinning/isolation, repetitions, documented settings, and reproducible scripts.
AI-use transparency and pitfall analysis	15	Honest AI-use note and meaningful benchmark-pitfall demonstration.
Report quality	15	Readable plots, concise interpretation, and thoughtful limitations.

Project #3: SSD Performance Profiling

Modern SSDs, especially NVMe SSDs, can deliver very high IOPS and throughput when requests are issued concurrently. Like memory, storage exhibits a throughput-latency tradeoff governed by queueing: increasing queue depth improves utilization until a saturation knee, after which latency rises sharply with little additional throughput.

Data-loss Warning
Never benchmark against a partition that contains valuable data. Use a dedicated empty partition, raw device reserved for tests, or a large disposable file with direct I/O. You are responsible for preventing data loss.

Learning Goals

- Measure QD=1 latency for reads and writes under random 4KiB and sequential 128KiB patterns.
- Measure maximum small-block IOPS and large-block sequential throughput.
- Reveal throughput-latency behavior as queue depth and/or parallelism increases.
- Explain block-size, access-pattern, read/write mix, working-set/LBA-range, and device-state effects.
- Characterize p95/p99/p99.9 tail latency and relate it to queueing and device state.

Tools

- FIO for controlled storage workloads: block size, queue depth, numjobs, direct I/O, patterns, mixes, and percentiles.
- Optional: OS disk statistics and SMART/health tools for temperature, throttling, and write-count observations.

Experimental Knobs

- Block size: 4KiB, 16KiB, 32KiB, 64KiB, 128KiB, 256KiB; optionally 512KiB or 1MiB for sequential.
- Access pattern: sequential vs. random; maintain alignment for random tests.
- Read/write ratio: 100% read, 100% write, 70/30, and 50/50.
- Queue depth and parallelism: sweep iodepth and/or numjobs, e.g., 1 to 256 as device supports.
- Data pattern: incompressible vs. compressible, held consistent and documented.
- Device state: fresh/burst vs. steady-state, temperature, SLC cache exhaustion, and background garbage collection.

Required Experiments and Plots

- Zero-queue baselines: QD=1 latency for 4KiB random read/write and 128KiB sequential read/write; include p95/p99.
- Block-size sweep: throughput and latency from the same runs for random and sequential patterns.
- Read/write mix sweep under fixed block size and pattern.
- Queue-depth/parallelism sweep: single throughput-vs-latency curve, knee identification, and Little's Law discussion.
- Tail-latency characterization at mid-QD and near-knee QD.
- Working-set/LBA-range and device-state note: discuss burst vs. steady-state if observable.
- Demonstrate or explain one misleading setup, such as buffered I/O/page-cache effects, wrong effective queue depth, insufficient preconditioning, or thermal throttling.

Deliverables

- FIO configs, scripts, raw results, parsed data, and plotting code.
- Methodology: SSD model, interface, filesystem vs. raw device, direct I/O usage, capacity/range used, OS/kernel, CPU, temperature if available.
- Plots/tables with units, error bars when feasible, and clear labels for fixed vs. varied knobs.
- Analysis grounded in queueing theory, controller behavior, interface ceilings, and tail-latency implications.

Grading Rubric

Total: 185 points

Category	Points	Description
Safety and zero-queue baselines	30	Safe setup, direct I/O/alignment, and accurate QD=1 latency table.
Block-size and pattern sweeps	35	Complete matrix and clear IOPS/MB/s vs latency interpretation.
Read/write and queue-depth analysis	40	Correct mix and QD experiments, knee identification, Little's Law reasoning.
Tail latency and device state	25	p50/p95/p99/p99.9 reporting and interpretation of queueing/thermal/GC effects.
Reproducibility and data hygiene	25	Configs, raw data, scripts, repeated trials, and environment documentation.
AI-use transparency and pitfall analysis	15	Honest AI-use note and meaningful misleading-setup analysis.
Report quality	15	Clear figures, concise explanation, and thoughtful limitations.